

SpectraLang

Arquitetura da Linguagem e Processo de Compilação

Uma linguagem e toolchain implementada em Rust para cargas de **IA/ML** e para a construção nativa de **APIs e serviços** HTTP/event-driven — um único compilador, um único runtime, um único binário.

Apresentação técnica-comercial · CLI v0.2.6 · MIT

Slide 02 · O Problema: a costura que quebra

Times de IA escrevem o modelo em **Python** (PyTorch/JAX) e reescrevem a camada de serviço em **Go / Rust / Node**. Resultado: duas linguagens, dois toolchains, dois conjuntos de habilidades e uma fronteira de deployment propensa a divergência de tipos, de performance e de comportamento.

Etapa	Linguagem típica	Custo / risco
Treinamento / experimento	Python	Ecossistema maduro, mas lento em produção
Inferência / serving	Go, Rust, Node	Reimplementação + drift de lógica
Tooling (lint, fmt, pkg, LSP)	Fragmentado	N× integrações e atrito de DX

Tese da SpectraLang: a mesma linguagem que descreve o tensor deve descrever a requisição HTTP — treino e serving no mesmo arquivo, mesmo runtime, mesmo artefato de deployment.

Slide 03 · A Tese: uma linguagem, um runtime, um binário

Dois workstreams deliberadamente acoplados compartilham compilador, runtime e toolchain:

Workstream	Escopo
AI/ML core	Tensor-first na linguagem; tipos científicos; ops ciente de forma; autodiff reverso; framework ML (módulos, losses, etc)
API platform (spectra.api)	async/await de 1ª classe; primitivos HTTP tipados; routing, middleware, JSON, TLS, drivers de DB; observabilidade

Decisão arquitetural-chave: `spectra.api` é um **pacote versionado separado**, não parte do `std`, para que padrões web evoluam no próprio ritmo sem inflar cada build com TLS/drivers/exportadores.

Slide 04 · Panorama da Linguagem: familiar à primeira vista

Semântica tipo Rust (traits, match, dyn, &self) + ergonomia tipo Python (import as, elif, f-strings) + API estilo PyTorch. Originais: **unless**, **module** obrigatório, visibilidade pub/internal/privada.

```
module demo;
import std.io;
import { println } from std.io;

let x: int = 10;          // explicito
let y = 20;              // inferido
let msg = f"Soma = {x + y}";

fn double(x: int) => int { x * 2 } // retorno implícito

fn classify(v: int) => int {
  unless v < 0 {
    switch v {
      case 0 => { return 10; }
      else  => { return 30; }
    }
  } else { return -10; }
}

enum State { Ready(int), Serving, Failed(int) }
match s {
  State::Ready(n) => n,
  State::Serving  => 100,
  _               => 0,
}
```

Fonte: SYNTAX_SUMMARY.md, tests/validation/120_stable_promoted_control_flow.spectra

Slide 05 · Diferencial 1: tensores no sistema de tipos

dtype, rank, dimensões, layout de memória e dispositivo são parte do tipo e verificados em tempo de compilação (códigos E1401–E1406). PyTorch pega o erro de forma no época 3; SpectraLang pega antes de rodar.

```
fn total(values: Tensor<float, rank1, dim4, row_major, cpu>) -> int {
    return tensor.sum_f(values) as int;
}

let w: Tensor<float, rank2, dim2, dim1, row_major, cpu> = [[2.0], [2.0]];
let loss: Tensor<float, rank0> = diff {           // região diferenciável nativa
    tensor.sum_t(tensor.mul(v, v))
};
let grad = tensor.grad(v);                       // autodiff de 1ª classe
```

Fonte: `tests/validation/102_pattern_tensor_ai_composition_stress.spectra`

`diff { }` é um **construct de linguagem** que rebaixa para `std.tensor.backward` — autodiff é recurso do compilador, não biblioteca de gravação de tape.

Slide 06 · Diferencial 2: treinar e servir em um arquivo

O mesmo arquivo .spectra treina o modelo e sobe o servidor de inferência — 67 linhas, sem costura de linguagem.

```
import std.tensor as tensor;
import std.ml as ml;
import std.serve as serve;

fn train_step(x, target, w, b) -&gt; int {
    let pred = ml.linear(x, w, b);
    let loss = ml.mse_loss(pred, target);
    tensor.backward(loss);
    ml.sgd_step(w, 0.1); ml.sgd_step(b, 0.1);
    return 0;
}

pub fn main() -&gt; int {
    tensor.set_grad_enabled(true);
    let model = ml.module_new();
    let dataset = ml.dataset_from_tensors(x, target, 4);
    let loader = ml.dataloader_new(dataset, 2, 2026); // seed =&gt; reprodutível
    // ... loop de treino ...
    let server = serve.server_new(2);
    serve.server_warmup(server);
    serve.server_enqueue(server, 21);
    serve.server_process_batch(server, 1);
    return 0;
}
```

Fonte: `examples/ai/mlp_training_serving.spectra`

Slide 07 · Diferencial 3: async/await + reactor de plataforma

Async é conceito de **primeira classe na IR** (AsyncSuspend/Resume/Ready, `Type::Task`), rebaixado para uma máquina de estados SSA. O runtime usa mio mapeando para **epoll** (Linux) / **IOCP** (Windows) / **kqueue** (macOS).

```
async fn add_one() -> int {
    let value = await ready_value(); // await prefixado
    return value + 1;
}

async fn from_block() -> int {
    let task: Task<int> = async { // bloco async -> Task<T>
        let b = await ready_value();
        b + 1
    };
    return await task;
}
```

Fonte: `tests/validation/121_async_await_lowering.spectra`

20 códigos de diagnóstico E2101–E2120 garantem segurança de async no compilador (valores não-Send através de `await`, traits `async` não object-safe, etc.).

Slide 08 · Arquitetura Geral: quatro crates, um seam

Workspace Cargo (9 crates). O trait `BackendDriver` é a dobradura arquitetural: `spectra-compiler` tem **zero** dependência de `Cranelift/runtime` (só `serde_json`), logo é embutível — é assim que o LSP o reusa.



tools/: `spectra-cli` (binário `spectralang`), `spectra-lsp`, `spectra-interop`. *Pacotes*: `spectra-api`, `spectra-db`.

Slide 09 · Front-end: lexer -> parser -> AST -> semântica -> lint

- Lexer: autômato escrito à mão (tokens com spans byte+linha); f-strings, comentários de bloco, operadores compostos.
- Parser: descida recursiva, 1 token de lookahead, com recuperação de erro (synchronize) que reporta múltiplos erros por passada.
- ModuleLoader: cache incremental — hash(source+features) evita re-lexar/re-parser em hit.
- Análise Semântica: visitor multipassada (imports -> declarações -> corpos -> genéricos -> tipos de MethodCall); symbol stack + ModuleRegistry compartilhado.
- Lint: 3 regras (unused-binding, unreachable-code, shadowing); regra 'deny' vira erro de compilação.

.spectra

```
■  
[1] LEXING      compiler/src/lexer      -&gt; Vec<Token>  
[2] PARSING     compiler/src/parser    -&gt; Module (AST)  
[3] SEMANTIC    compiler/src/semantic  -&gt; Vec<SemanticError>  
[4] LINT        compiler/src/lint      -&gt; Vec<LintDiagnostic>  
■  
BackendDriver::run(&ast, &options)
```

Slide 10 · Mid-end: IR SSA (SIR) + autodiff + otimização

Após o front-end, a AST rebaixa para uma **IR SSA** (SIR) e, em paralelo, para um **TensorGraph** de mais alto nível usado em fusão e legalização CPU/WGPU. Autodiff é nó nativo da IR.

- Lowering (AST->IR): escopo SSA, aloca para vars mutáveis, monomorfização de genéricos com name mangling e teto de 512 especializações.
- Autodiff: `InstructionKind::AutodiffStep` (contrato fechado) materializado por `materialize_autodiff_steps()` antes da verificação.
- Passes (opt-level gated): `ConstantFolding` (≥ 1), `FunctionInlining` + `DCE` (≥ 2), `ConcurrentSpawnJoinFusion` (se optimize).
- `LoopStructureValidation` roda sempre; `verify_module()` roda antes e depois das otimizações (verificação dupla).

IR carrega debug info: `SourceSpan` por instrução e `LocalDebugInfo` por local — habilita stack traces nível código-fonte.

Slide 11 · Back-end: Cranelift JIT + AOT

- JIT (caminho primário): JITBuilder com optimizer 'speed' (padrão é none); declara/define funções via FunctionBuilder; PHI -> block parameters do Cranelift.
- AOT: cranelift-object emite COFF/ELF/Mach-O; --emit-exe sintetiza shim main(argc,argv) que sobe o runtime e chama o entry point.
- Host calls: ponte entre código nativo JIT e Rust via ABI SpectraHostValue + registro de funções; fast-paths no_mangle para string/map/concurrent.
- Debug: mapas JSON de debug + DWARF/CodeView para gdb/lldb/cdb■ --dump-ir imprime a IR textual.

```
[12a] JIT    backend/src/codegen.rs    CodeGenerator -&gt; JITModule -&gt; memória
[12b] AOT    backend/src/aot.rs        AotCodeGenerator -&gt; .o/.obj -&gt; linker
                                   (requer libspectra_runtime.a)
```

Slide 12 · Runtime: memória híbrida, reactor, stdlib, api

- HybridMemory: GC tracado (`Gc<T>`, `GcRoot<T>`) para valores gerenciados + frames manuais (`manual_frame_enter/exit`) para scratch do JIT.
- Host-call ABI estável: registro process-wide (mutex); status codes `SUCCESS/INVALID_ARG/NOT_FOUND/INTERNAL`.
- stdlib registrada em `runtime::initialize()` via `register_standard_library()` + `spectra_api::register()` (contrato de 211 host calls tipados).
- Reactor async sobre mio + Waker + fila de prioridade; exposto via `spectra.async.reactor.*`.

spectra.api (211 host calls assereados em compile-time entre `runtime/src/api/mod.rs` e o crate `spectra-api`): `http`, `server`, `client`, `json`, `tls`, `routing`, `middleware`, `db` (`sqlite/postgres/redis`), `trace`, `metrics`, `health`.

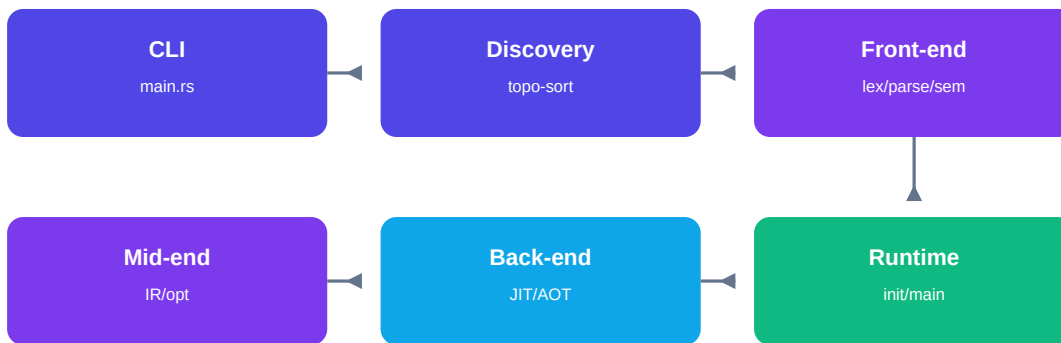
Slide 13 · Toolchain completo em um único binário

Tudo em spectralang — sem instalar formatter, linter, package manager ou test runner separados.

Comando	Propósito
run / compile / check	Execução JIT, compilação, type-check
lint	3 regras; --allow/--deny
fmt	Formatador (inclusive --stdin)
repl / new	REPL interativo; scaffolder de projeto
package (19 subcmds)	lock, build, test, add, publish, catalog...
db	Migrações de banco (apply/inspect/rollback)
--json / --sarif	Diagnósticos SARIF 2.1.0 p/ GitHub Code Scanning

LSP spectra-lsp: 14 capacidades (hover, go-to-def, rename, completion, inlay hints, semantic tokens, quickfix...). Níveis -00 . . -03.

Slide 14 · Processo de Compilação ponta-a-ponta: spectralang run



- Resolução de projeto (spectra.toml) e descoberta de fontes; `ProjectPlan::topological_order()` ordena por imports (detecção de ciclo).
- Synthetic 'module <nome>;' inserido se ausente; compilação módulo a módulo na ordem topológica.
- `CompilationPipeline::compile()` (front-end) -> `FullPipelineBackend::run()` (mid+back) -> `execute_artifacts()`.
- Runtime: `initialize()` + `register_standard_library()` + `spectra_api::register()`; `codegen.execute_entry_point("main", ir)`; código de saída propagado.

Fonte: `tools/spectra-cli/src/compiler_integration.rs`, `main.rs`

Slide 15 · Evidências & Maturidade

Benchmarks cross-linguagem (31 cenários × Spectra/Go/Java/Rust, 3 warmups, 20 amostras, drift <=15%):

Cenário	vs Go	vs Rust
tensor-reduce	1.05x (paridade)	1.6x
tensor-elementwise	1.2x	2.0x
tensor-matmul	1.94x	2.5x
cpu-hashmap	6.0x	6.9x
cpu-string-build	71.7x (gap conhecido)	66.9x

Roadmap: ~67% concluído (180/267 itens). Fases 0–22 prontas (AI/ML core maduro, async core, API foundation). Fronteira ativa: middleware/segurança (23), recursos avançados de API (24), DB (25), observabilidade (27), conformance v1.0 (28).

Status honesto: ainda não é v1.0 estável; JIT é primário; exe standalone não totalmente integrado ponta-a-ponta; stdlib incompleta vs plano.

Slide 16 · Considerações Finais

Por que SpectraLang merece atenção:

- Tensores e requisições HTTP são **tipos de primeira classe** — erros de forma viram erro de compilação.
- Treino e serving no **mesmo arquivo/runtime/artefato**.
- Autodiff e async são **recursos do compilador**, não bibliotecas.
- JIT **e** AOT pela mesma pipeline; Cranelift em todas as platforms.
- Toolchain completo em **um binário** com diagnósticos SARIF e LSP de 14 capacidades.
- Cultura **evidence-driven**: claims de performance exigem artefatos de benchmark reproduzíveis.

Posicionamento: projeto credível e disciplinado, com núcleo de IA maduro e plataforma de API em franca evolução — não uma promessa exagerada, e sim uma base de engenharia sólida para cargas de ML e serviços.

SpectraLang · CLI v0.2.6 · MIT · compilador/runtime/toolchain em Rust